Laboratorio de Replicación Física y Lógica en PostgreSQL

Empresa: LogiTrans S.A.
Curso: Bases de Datos II
Estudiante: Justin Mayorga Hernández
Fecha: 30 de octubre de 2026

1. Introducción

El presente laboratorio tiene como objetivo implementar y validar un clúster de PostgreSQL que combine replicación física y replicación lógica, utilizando tres instancias independientes del motor de base de datos.

La solución se desarrolló mediante PostgreSQL 18.6, Docker Compose y scripts de automatización en PowerShell, Bash y SQL. Se implementaron mecanismos de replicación, configuraciones de seguridad y pruebas automatizadas para verificar el correcto funcionamiento del sistema.

2. Arquitectura implementada

El clúster está compuesto por los siguientes nodos:

Nodo	Puerto del host	Función
central-db	15432	Servidor primario de producción
regional-db	5433	Servidor standby con replicación física
bi-db	5434	Servidor de reportes con replicación lógica

La arquitectura implementada sigue el flujo:

central-db → replicación física → regional-db → replicación lógica → bi-db

La replicación física permite mantener en regional-db una copia del clúster principal, incluyendo las tablas empleados y entregas.

Por su parte, la replicación lógica permite enviar únicamente la tabla empleados al nodo bi-db, separando los datos necesarios para reportes de los demás datos operativos.

3. Evidencias de funcionamiento

Evidencia 1. Estado de los tres nodos

```
PS C:\Users\justin\DEsktop\Lab_Bases> cd "C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres"
PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres> docker compose ps
NAME                          IMAGE
     COMMAND                  SERVICE       CREATED       STATUS                 PORTS
logitrans_lab-bi-db-1         postgres:18-bookworm@sha256:5c4783a8cc3f007491216 4f94dfa1be4e043438f0f707cf968d7af9a0d31a2
c8   "docker-entrypoint.s…"   bi-db         5 hours ago   Up 5 hours (healthy)   127.0.0.1:5434->5432/tcp
logitrans_lab-central-db-1    postgres:18-bookworm@sha256:5c4783a8cc3f007491216 4f94dfa1be4e043438f0f707cf968d7af9a0d31a2
c8   "docker-entrypoint.s…"   central-db    5 hours ago   Up 5 hours (healthy)   127.0.0.1:15432->5432/tcp
logitrans_lab-regional-db-1   postgres:18-bookworm@sha256:5c4783a8cc3f007491216 4f94dfa1be4e043438f0f707cf968d7af9a0d31a2
c8   "bash /lab/standby.s…"   regional-db   5 hours ago   Up 5 hours (healthy)   127.0.0.1:5433->5432/tcp
PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres>
```

Se ejecutó el comando docker compose ps para consultar el estado de los tres contenedores que conforman el clúster.

Esta verificación permite comprobar que central-db, regional-db y bi-db se encuentran en ejecución y que sus comprobaciones de salud reportan un estado satisfactorio.

Evidencia 2. Replicación física entre central-db y regional-db

```
PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres> docker compose exec -T -u postgres central-db psql -d logitrans
 -c "SELECT application_name, state FROM pg_stat_replication;"
 application_name |   state
------------------+-----------
 regional-db      | streaming
(1 row)

PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres>
PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres> docker compose exec -T -u postgres regional-db psql -d logitran
s -c "SELECT pg_is_in_recovery(); SELECT status, sender_host, slot_name FROM pg_stat_wal_receiver;"
 pg_is_in_recovery
-------------------
 t
(1 row)

  status   | sender_host |     slot_name
-----------+-------------+-------------------
 streaming | central-db  | regional_physical
(1 row)

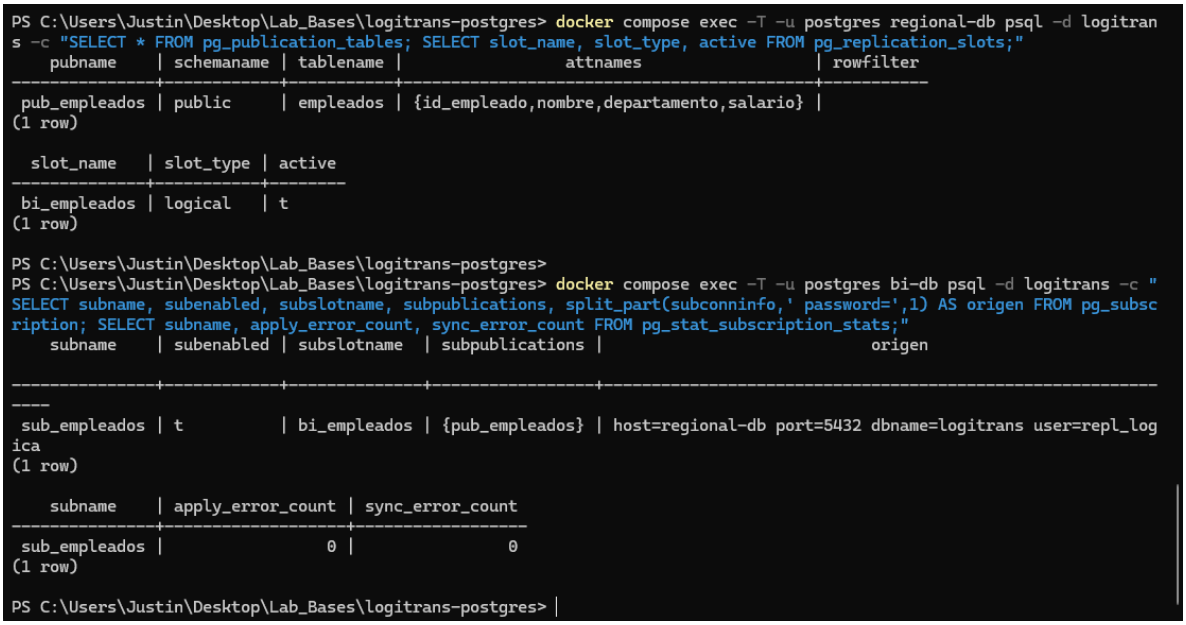
PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres>
```

Se realizaron consultas a las vistas pg_stat_replication y pg_stat_wal_receiver, además de verificar la función pg_is_in_recovery().

Estas consultas permiten identificar al servidor primario, confirmar que regional-db opera como standby y comprobar que existe una conexión activa para la transmisión de registros WAL.

El estado streaming evidencia el funcionamiento de la conexión de replicación física.

Evidencia 3. Replicación lógica entre regional-db y bi-db



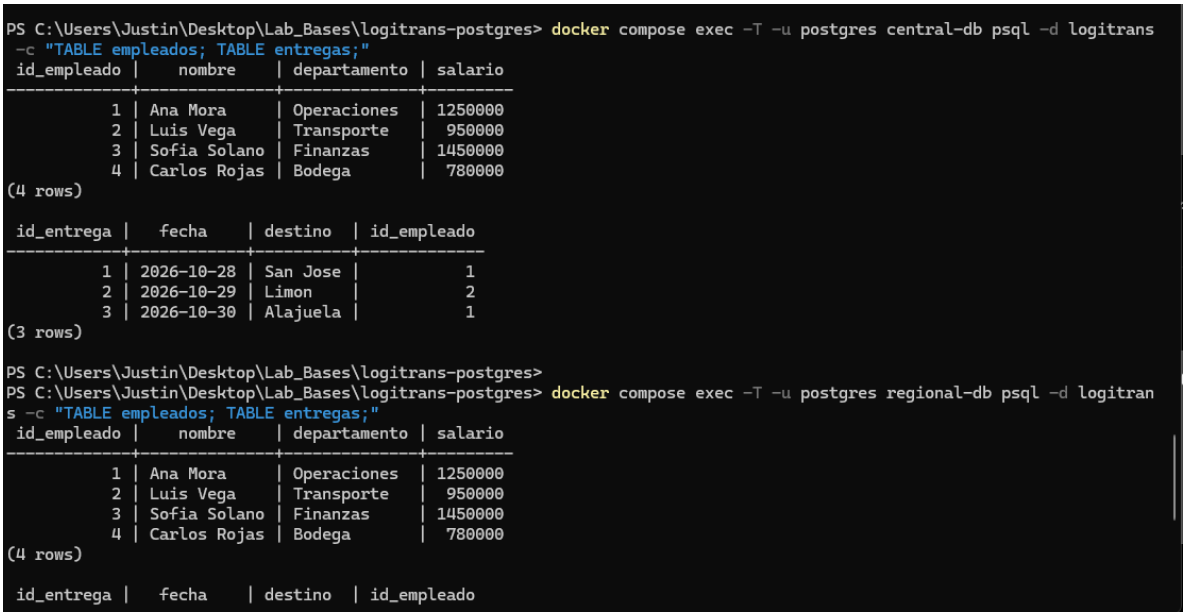
Se consultaron las publicaciones, los slots de replicación y la configuración de la suscripción lógica.

La publicación pub_empleados incluye únicamente la tabla empleados, mientras que la suscripción sub_empleados utiliza regional-db como servidor de origen.

Esta configuración permite que los cambios de la tabla publicada sean transmitidos al nodo de reportes sin replicar la tabla entregas.

También se comprobaron los indicadores de funcionamiento y los contadores de errores asociados a la suscripción.

Evidencia 4. Validación de datos replicados

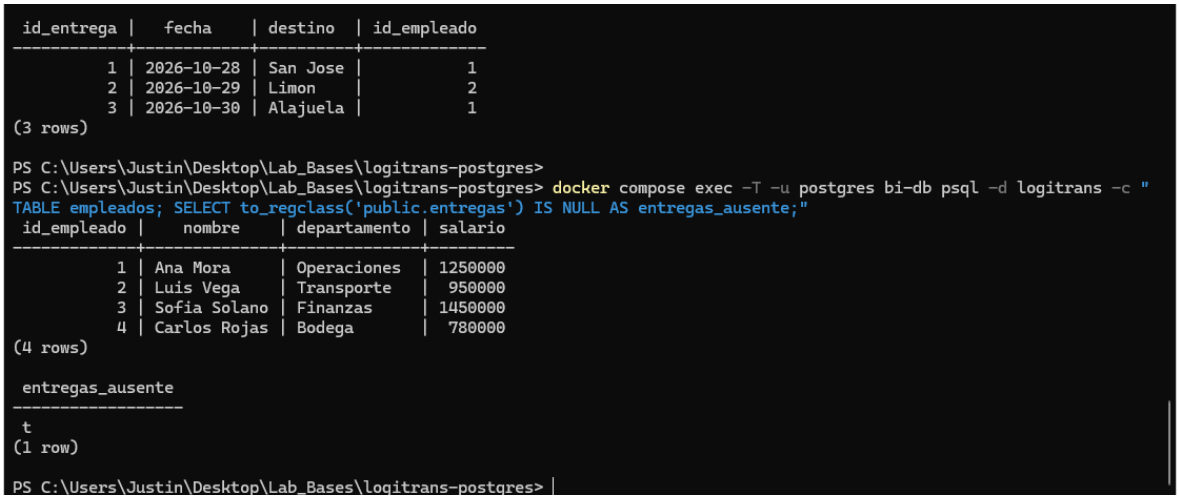


```
PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres> docker compose exec -T -u postgres central-db psql -d logitrans
 -c "TABLE empleados; TABLE entregas;"
 id_empleado |    nombre    | departamento | salario
-------------+--------------+--------------+---------
           1 | Ana Mora     | Operaciones  | 1250000
           2 | Luis Vega    | Transporte   |  950000
           3 | Sofia Solano | Finanzas     | 1450000
           4 | Carlos Rojas | Bodega       |  780000
(4 rows)

 id_entrega |   fecha    | destino  | id_empleado
------------+------------+----------+-------------
          1 | 2026-10-28 | San Jose |           1
          2 | 2026-10-29 | Limon    |           2
          3 | 2026-10-30 | Alajuela |           1
(3 rows)

PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres>
PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres> docker compose exec -T -u postgres regional-db psql -d logitran
s -c "TABLE empleados; TABLE entregas;"
 id_empleado |    nombre    | departamento | salario
-------------+--------------+--------------+---------
           1 | Ana Mora     | Operaciones  | 1250000
           2 | Luis Vega    | Transporte   |  950000
           3 | Sofia Solano | Finanzas     | 1450000
           4 | Carlos Rojas | Bodega       |  780000
(4 rows)

 id_entrega |   fecha    | destino  | id_empleado
```



```
 id_entrega |   fecha    | destino  | id_empleado
------------+------------+----------+-------------
          1 | 2026-10-28 | San Jose |           1
          2 | 2026-10-29 | Limon    |           2
          3 | 2026-10-30 | Alajuela |           1
(3 rows)

PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres>
PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres> docker compose exec -T -u postgres bi-db psql -d logitrans -c "
TABLE empleados; SELECT to_regclass('public.entregas') IS NULL AS entregas_ausente;"
 id_empleado |    nombre    | departamento | salario
-------------+--------------+--------------+---------
           1 | Ana Mora     | Operaciones  | 1250000
           2 | Luis Vega    | Transporte   |  950000
           3 | Sofia Solano | Finanzas     | 1450000
           4 | Carlos Rojas | Bodega       |  780000
(4 rows)

 entregas_ausente
------------------
 t
(1 row)

PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres>
```

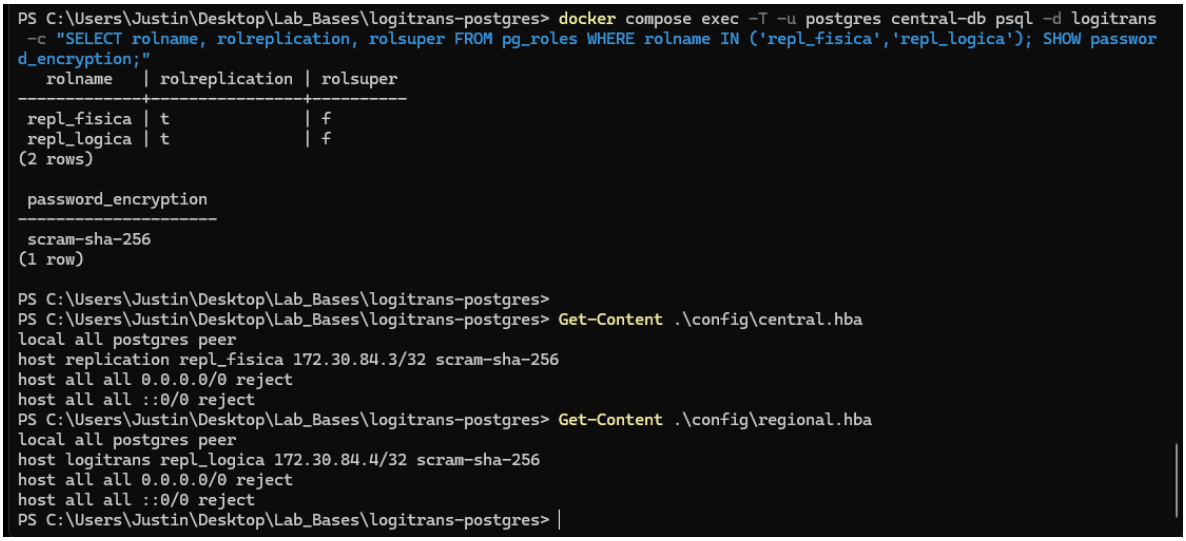
Se ejecutaron consultas SQL sobre las tres instancias para comparar los datos disponibles.

En central-db y regional-db se verificaron las tablas empleados y entregas, mientras que en bi-db se consultó la tabla empleados y se comprobó la ausencia de entregas.

Adicionalmente, las pruebas automatizadas verificaron operaciones de inserción, modificación y eliminación de registros, comparando los valores replicados en los nodos correspondientes.

Esta evidencia permite comprobar tanto la replicación de los datos como la separación de información destinada al área de reportes.

Evidencia 5. Roles y parámetros de seguridad

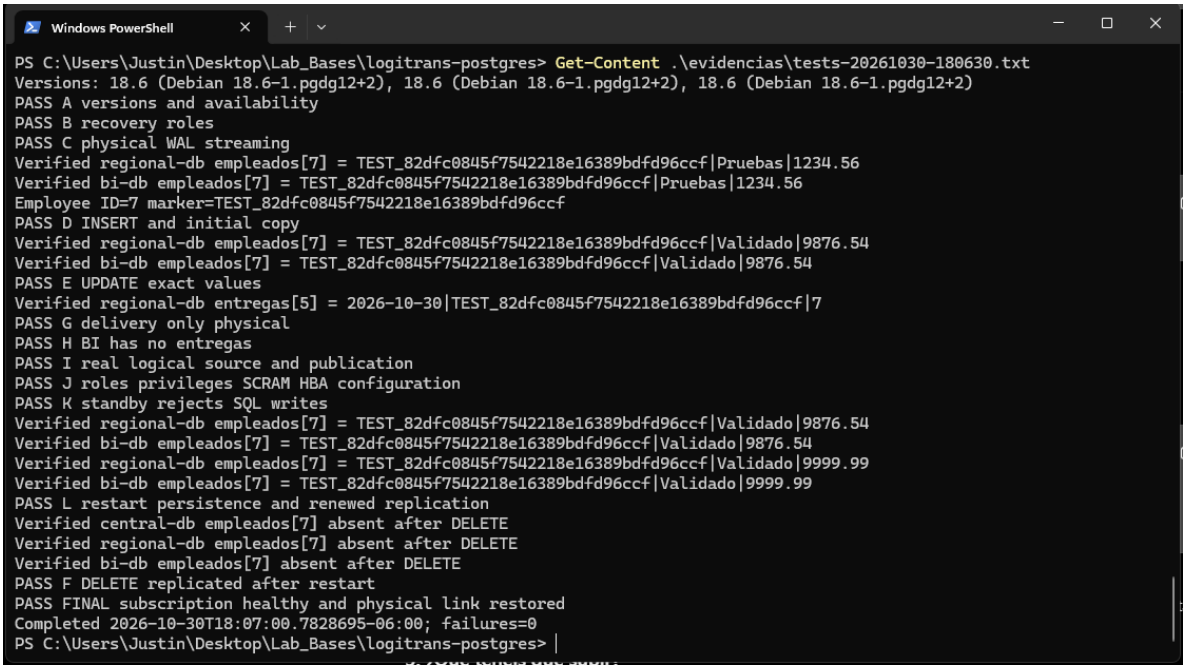


Se verificaron los roles utilizados para la replicación, sus privilegios y los mecanismos de autenticación configurados.

La implementación utiliza roles específicos para las conexiones de replicación física y lógica, evitando otorgarles privilegios de superusuario.

También se configuraron reglas de acceso mediante pg_hba.conf y autenticación SCRAM-SHA-256, restringiendo las conexiones a los nodos autorizados.

Evidencia 6. Resultados de las pruebas automatizadas



Se implementó un conjunto de pruebas automatizadas mediante el script lab.ps1.

Las comprobaciones abarcan disponibilidad de los nodos, identificación del servidor primario y standby, replicación física y lógica, operaciones sobre los datos, separación de tablas, privilegios de seguridad y persistencia después de reiniciar los contenedores.

En las ejecuciones documentadas se obtuvieron resultados satisfactorios para las pruebas A–L, además de la comprobación final de estado.

Los resultados detallados se almacenaron en archivos de texto dentro del directorio evidencias/, permitiendo revisar los valores comprobados y las operaciones ejecutadas.

Evidencia 7. Configuración de la replicación física

Se presentan los parámetros de configuración de PostgreSQL utilizados para habilitar la replicación y el script de inicialización del nodo regional. La configuración permite generar y transmitir registros WAL, mientras que pg_basebackup crea una copia física del clúster primario y prepara el nodo secundario para funcionar como standby.

```
PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres> Get-Content .\config\postgresql.conf
listen_addresses = '*'
password_encryption = 'scram-sha-256'
wal_level = logical
max_wal_senders = 10
max_replication_slots = 10
max_worker_processes = 16
max_logical_replication_workers = 8
max_sync_workers_per_subscription = 2
hot_standby = on
wal_receiver_status_interval = 1s
wal_sender_timeout = 30s
wal_keep_size = 128MB
max_slot_wal_keep_size = 1GB
log_line_prefix = '%m [%p] %u@%d '
log_connections = 'authentication'
log_statement = none
log_min_error_statement = panic
```

```
PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres> Get-Content .\scripts\standby.sh
#!/bin/bash
set -Eeuo pipefail
mkdir -p "$PGDATA"
chown postgres:postgres "$PGDATA"
chmod 700 "$PGDATA"
# Password file is outside PGDATA and regenerated at every start; no secret in conninfo.
printf 'central-db:5432:replication:repl_fisica:%s\n' "$(cat /run/secrets/physical)" > /var/lib/postgresql/physical.pgpa
ss
chown postgres:postgres /var/lib/postgresql/physical.pgpass
chmod 600 /var/lib/postgresql/physical.pgpass
if [ ! -s "$PGDATA/PG_VERSION" ]; then
  if [ -n "$(ls -A "$PGDATA")" ]; then
    echo 'Incomplete PGDATA: manual review required; refusing to delete files.' >&2
    exit 1
  fi
  gosu postgres pg_basebackup \
    --dbname='host=central-db user=repl_fisica application_name=regional-db passfile=/var/lib/postgresql/physical.pgpass
' \
    -D "$PGDATA" -R -X stream -S regional_physical --checkpoint=fast --no-password
fi
test -f "$PGDATA/standby.signal" || { echo 'standby.signal missing; refusing writable startup' >&2; exit 1; }
exec /usr/local/bin/docker-entrypoint.sh "$@"
PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres>
```

Evidencia 8. Configuración de la replicación lógica

Se muestra la definición de la publicación lógica pub_empleados, que incluye exclusivamente la tabla empleados, y el script que configura la suscripción sub_empleados desde el nodo de reportes. La conexión utiliza regional-db como origen, manteniendo la topología de replicación requerida.

```
PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres> Get-Content .\sql\central.sql
CREATE ROLE repl_fisica LOGIN REPLICATION PASSWORD :'physical';
CREATE ROLE repl_logica LOGIN REPLICATION PASSWORD :'logical';
REVOKE CONNECT, TEMPORARY ON DATABASE logitrans FROM PUBLIC;
GRANT CONNECT ON DATABASE logitrans TO repl_logica;
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
CREATE TABLE empleados (
 id_empleado SERIAL PRIMARY KEY, nombre TEXT NOT NULL,
 departamento TEXT, salario NUMERIC
);
CREATE TABLE entregas (
 id_entrega SERIAL PRIMARY KEY, fecha DATE, destino TEXT,
 id_empleado INT REFERENCES empleados(id_empleado)
);
GRANT USAGE ON SCHEMA public TO repl_logica;
GRANT SELECT ON empleados TO repl_logica;
INSERT INTO empleados(nombre,departamento,salario) VALUES
 ('Ana Mora','Operaciones',1250000),('Luis Vega','Transporte',950000),
 ('Sofia Solano','Finanzas',1450000),('Carlos Rojas','Bodega',780000);
INSERT INTO entregas(fecha,destino,id_empleado) VALUES
 ('2026-10-28','San Jose',1),('2026-10-29','Limon',2),('2026-10-30','Alajuela',1);
CREATE PUBLICATION pub_empleados FOR TABLE public.empleados;
SELECT pg_create_physical_replication_slot('regional_physical', true);
```

```
PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres> Get-Content .\scripts\subscribe.sh
#!/bin/bash
set -Eeuo pipefail
if [ "$(psql -XAt -U postgres -d logitrans -c "SELECT count(*) FROM pg_subscription WHERE subname='sub_empleados'")" = 0
 ]; then
  # Feed through stdin; never echo conninfo or include it in process arguments.
  psql -X -v ON_ERROR_STOP=1 -U postgres -d logitrans <<SQL
CREATE SUBSCRIPTION sub_empleados
 CONNECTION 'host=regional-db port=5432 dbname=logitrans user=repl_logica password=$(cat /run/secrets/logical) applicati
on_name=bi-empleados connect_timeout=5 options=-crow_security=off'
 PUBLICATION pub_empleados WITH (copy_data=true, slot_name='bi_empleados', failover=false);
SQL
fi
PS C:\Users\Justin\Desktop\Lab_Bases\logitrans-postgres>
```

4. Documentación de la implementación

La implementación completa del laboratorio se encuentra disponible en el repositorio:

https://github.com/JusMH/Lab_Bases

Dentro del directorio logitrans-postgres se incluyen los archivos de configuración de PostgreSQL, Docker Compose, los scripts SQL, los mecanismos de automatización y la documentación técnica correspondiente.

El presente informe reúne las evidencias de ejecución obtenidas durante las pruebas del laboratorio, con el propósito de demostrar el funcionamiento de la replicación física y lógica, las configuraciones de seguridad y la correcta distribución de los datos entre los tres nodos.

5. Conclusiones

Las pruebas ejecutadas permitieron verificar la transmisión de datos, la separación de información entre los nodos, el uso de roles específicos y la continuidad de la replicación después de reiniciar los servicios.

Se consiguió así un entorno de laboratorio reproducible que demuestra el funcionamiento conjunto de ambos mecanismos de replicación de PostgreSQL.